Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Лабораторная работа №2

«Проектирование архитектуры и микроархитектуры нейросетевого процессора»

по дисциплине «Организация вычислительных систем»

Выполнил:
студент группы P4130
Дворкин Борис Александрович

Преподаватель:
Быковский Сергей Вячеславович

Санкт-Петербург
2026 г.

Содержание

1. Задание 3

2. Структура сети (п. 4.1) 3

3. Архитектура процессора (п. 4.2) 4

3.1. Обозначения на схемах 4

3.2. Варианты распараллеливания и конвейеризации 5

3.3. Структурная схема 6

3.4. Модель исполнения: поток, варп, линия, маска 7

3.5. Организация памяти 8

3.6. Модель вычислений, синхронизация и программирование 9

4. Микроархитектура функциональных узлов (п. 4.3) 10

4.1. Вычислительное ядро PE 10

4.2. Блок сигмоиды SIG 12

4.3. Bus matrix и контроллер ввода-вывода 13

5. Форматы данных и программирование (п. 4.4) 14

5.1. Форматы чисел 14

5.2. Формат команды и система команд 15

5.3. Описание сети и процесс конфигурирования 16

5.4. Расчёт сетей другой структуры 18

5.5. Что программируется и как 19

6. Алгоритм расчёта выхода сети (п. 4.5) 20

6.1. Распределение работы 20

6.2. Программа варпа 21

6.3. Пример: изображение круга 23

6.4. Диаграмма последовательности 23

7. Оценка времени вычисления выхода сети (п. 4.6) 26

7.1. Способы сокращения задержки 27

8. Загрузка вычислительных ядер (п. 4.7) 28

9. Выводы 29

1. Задание

Разработать архитектуру вычислителя для расчёта выхода нейронной сети из лабораторной работы №1, микроархитектуру его функциональных узлов, формат описания сети и алгоритм конфигурирования процессора. Рассмотреть варианты распараллеливания и конвейеризации. В отчёте описать структуру сети, архитектуру процессора, микроархитектуру узлов, форматы данных и процесс конфигурирования, алгоритм расчёта выхода сети, оценку времени расчёта и загрузку каждого вычислительного ядра.

Требования: от 3 до 7 вычислительных ядер (PE), равномерно загруженных; коммуникационная среда между узлами; контроллер ввода-вывода, принимающий входные данные и выводящий результат в консоль оператора; локальная память у каждого ядра и общая память.

Принятые проектные решения: 4 вычислительных ядра с моделью исполнения SIMT (одна команда выполняется группой потоков, как в потоковых мультипроцессорах NVIDIA); хранение весов в формате BF16, вычисления в FP32. Расчётное время вычисления выхода сети: 238 тактов. Все такты в отчёте получены программой `tools/processor_timing_model.py`, которая пошагово выполняет расписание команд по модели задержек из раздела 7.

2. Структура сети (п. 4.1)

Сеть взята из лабораторной работы №1 без изменений. Вход: изображение 7×7, пиксели пронумерованы построчно (номер пикселя j = 7 · строка + столбец), каждый пиксель равен 0 или 1. Сеть имеет три слоя нейронов: входной слой из 49 входов только передаёт пиксели и ничего не вычисляет; скрытый слой из 8 нейронов; выходной слой из 3 нейронов (круг, квадрат, треугольник). Вычисления выполняются в двух слоях: скрытом и выходном. Смещений (bias) нет. Выход скрытого нейрона i:

$$h_i = \sigma\left(\sum_{j=0}^{48} w_{ij}^{(1)}\, x_j\right), \quad i = 0,\ \ldots,\ 7$$

Выход выходного нейрона o:

$$y_o = \sigma\left(\sum_{i=0}^{7} w_{oi}^{(2)}\, h_i\right), \quad o = 0, 1, 2, \qquad \sigma(z) = 1/(1+e^{-z})$$

Обозначения: x_j: значение пикселя j; $w^{(1)}_{ij}$: вес связи от пикселя j к скрытому нейрону i (матрица W^1 размером 8 × 49); h_i: выход скрытого нейрона i; $w^{(2)}_{oi}$: вес связи от скрытого нейрона i к выходному нейрону o (матрица W^2 размером 3 × 8); y_o: выход выходного нейрона o; z: взвешенная сумма, аргумент сигмоиды; e: основание натурального логарифма.

Ответ сети: номер выхода с наибольшим значением y_o; если наибольшее значение меньше 0.5, ответ «шум» (класс 3). При равных значениях выбирается меньший номер.

Процессор выполняет только прямой проход. Сигмоида входит в прямой проход: без неё два слоя схлопнулись бы в одно умножение на матрицу W^2W^1, а веса из лабораторной работы 1 обучены именно с сигмоидой. Обратный проход (производная сигмоиды и поправки весов) нужен только для обучения; обучение выполнено в лабораторной работе 1, и в процессор веса загружаются готовыми.

Таблица 1. Объём вычислений на одно изображение

Слой	Умножений с накоплением	Сигмоид	Весов
Скрытый	8 · 49 = 392	8	392
Выходной	3 · 8 = 24	3	24
Всего	416	11	416

Рисунок 1 показывает тот же расчёт в виде схемы. Серые прямоугольники обозначают данные, которые хранятся (вход x, веса W^1 и W^2, промежуточный вектор h, выход y). Белые прямоугольники обозначают операции. Стрелки показывают, какие данные идут на вход каждой операции. Блок «Σ W^1·x» вычисляет 8 взвешенных сумм, каждая по 49 слагаемым; блок «σ» применяет сигмоиду к каждой сумме.

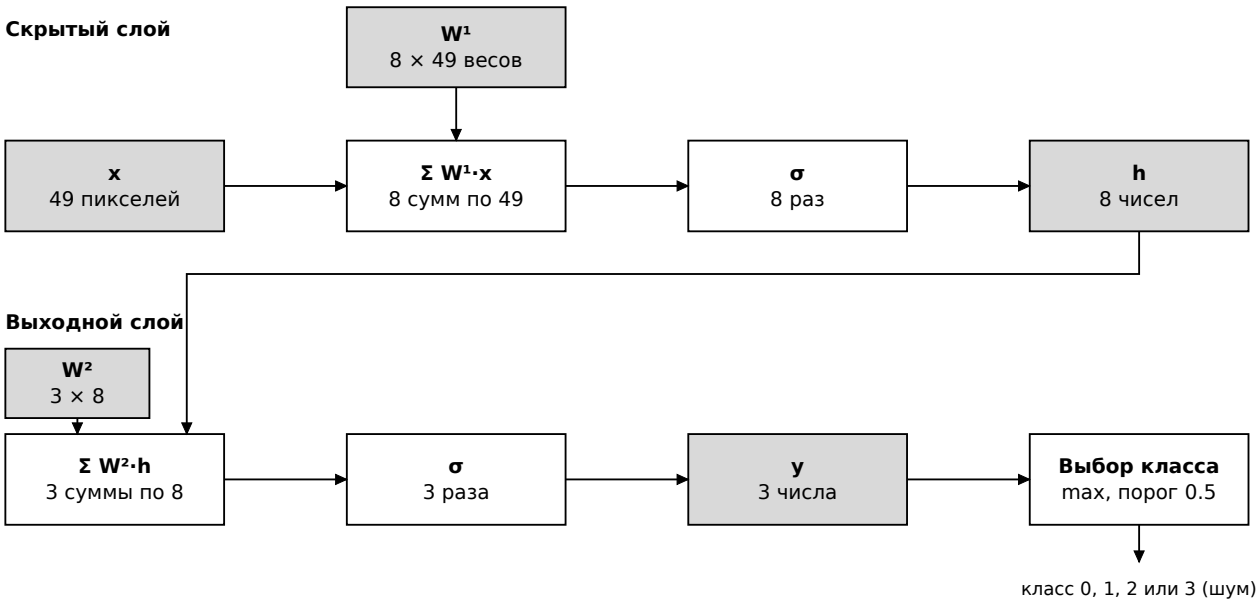


Рисунок 1. Прямой проход сети 49-8-3

3. Архитектура процессора (п. 4.2)

3.1. Обозначения на схемах

Все структурные схемы отчёта нарисованы по одним правилам, показанным на рисунке 2. Форма блока означает его тип, цвет различает только память (серый) и логику (белый). Линия со стрелкой означает передачу данных в направлении стрелки; стрелки на обоих концах означают передачу в обе стороны. Блок-схемы алгоритмов (рисунки 9 и 11) выполнены по ГОСТ 19.701-90: скруглённый прямоугольник обозначает начало или конец, прямоугольник действие, параллелограмм ввод или вывод данных, ромб проверку условия.

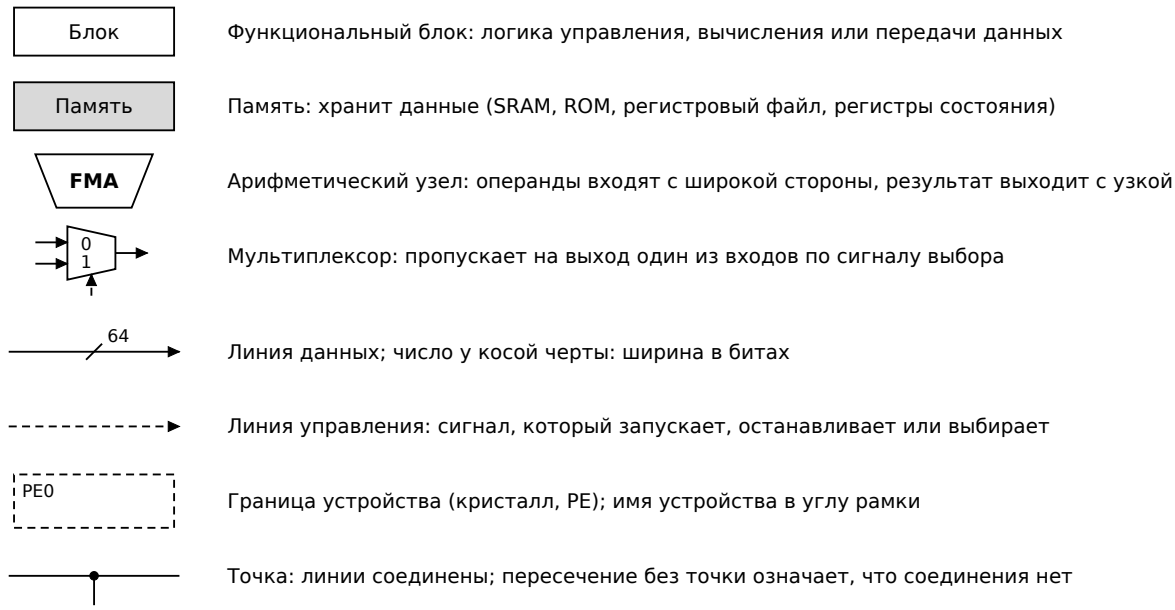


Рисунок 2. Обозначения на структурных схемах

3.2. Варианты распараллеливания и конвейеризации

Работу сети можно делить между исполнителями четырьмя способами (таблица 2). Исполнителем может быть ядро PE или поток внутри ядра.

Таблица 2. Способы распределения работы

Способ	**Что делится**	**Применимость к сети 49-8-3**
По изображениям	Каждое ядро считает своё изображение целиком	Не уменьшает время расчёта одного изображения; каждое ядро хранит все 416 весов
По слоям (конвейер слоёв)	Ядра считают разные слои, изображения идут друг за другом	Слои несоразмерны: 392 и 24 умножения; ядро выходного слоя простаивает большую часть времени
По нейронам слоя	Каждое ядро считает свои нейроны целиком	Выбран для скрытого слоя: 8 нейронов делятся на 4 ядра поровну, по 2 нейрона
По входам нейрона	Части одной взвешенной суммы считают разные исполнители, затем части складываются	Выбран внутри ядра: 49 входов нейрона делятся между 32 потоками; выбран для выходного слоя: каждое ядро считает вклад своих двух скрытых нейронов во все три выхода

Число ядер выбрано равным 4: из допустимых значений 3–7 только на 4 делятся поровну 8 нейронов скрытого слоя. При 3 ядрах распределение 3/3/2, при 5–7 ядрах часть ядер получает 2 нейрона, а часть 1, и время определяется ядрами с 2 нейронами.

Конвейеризация применена на трёх уровнях:

- внутри исполнительных блоков: умножитель-сумматор, загрузка из памяти и блок сигмоиды разбиты на ступени; новая команда выдаётся, пока предыдущая ещё проходит последние ступени;
- чередование варпов: пока одна группа потоков ждёт результата предыдущей команды, ядро выдаёт команду другой группы (раздел 7: 238 тактов с чередованием

против 297 без него);

- двойная буферизация входа: в общей памяти два буфера изображения; хост передаёт следующее изображение, пока ядра считают текущее.

Конвейер по слоям между ядрами не применяется по причине, указанной в таблице 2.

3.3. Структурная схема

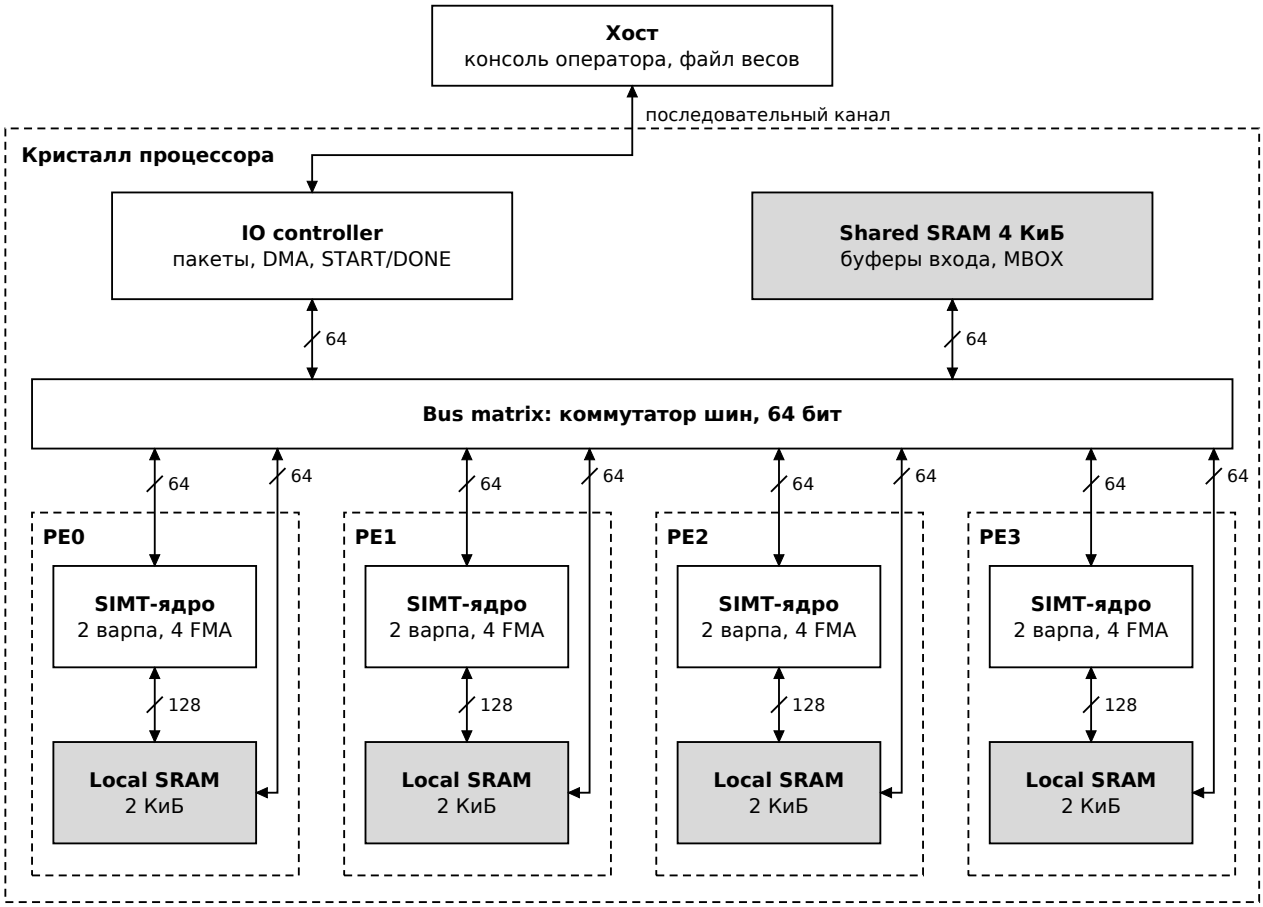


Рисунок 3. Структурная схема процессора

Как читать рисунок 3. Внешняя штриховая рамка обозначает границу кристалла; хост (компьютер оператора) находится вне кристалла и связан с ним последовательным каналом. Белые блоки обозначают логику, серые память. Каждое ядро PE0–PE3 обведено своей штриховой рамкой: внутри ядра находятся SIMT-ядро (вычислительная часть) и его локальная память. Числа у косых черт указывают ширину линий в битах.

Назначение блоков:

- **IO controller** (контроллер ввода-вывода): единственная связь кристалла с хостом. Принимает пакеты с хоста, раскладывает конфигурацию по памятям ядер, кладёт входное изображение в общую память и рассылает его копию в локальную память каждого ядра, подаёт сигнал START, принимает результат от PE0 и отправляет его на хост для вывода в консоль.
- **Bus matrix**: коммутатор шин шириной 64 бита. Соединяет ведущие устройства (IO controller, ядра) с ведомыми (общая память, локальные памяти ядер, регистры IO

controller). Несколько передач к разным ведомым идут одновременно; при обращении нескольких ведущих к одному ведомому арбитр пропускает их по очереди.

- **Shared SRAM** (общая память, 4 КиБ): память, доступная всем блокам через bus matrix. Хранит два буфера входного изображения и почтовый ящик MBOX, через который ядра передают ядру PE0 частичные суммы выходного слоя.
- **PE** (processing element, вычислительное ядро): SIMT-ядро с двумя группами по 32 потока и четырьмя умножителями-сумматорами FP32. Устройство ядра описано в разделе 4.1.
- **Local SRAM** (локальная память ядра, 2 КиБ): у каждого ядра своя. Хранит веса нейронов этого ядра и копию входного изображения. Имеет два порта: один для своего ядра, второй для bus matrix (через него IO controller записывает веса и изображение).

3.4. Модель исполнения: поток, варп, линия, маска

Ядро выполняет программу по правилам SIMT (single instruction, multiple threads: одна команда, много потоков).

- **Поток**: одно исполнение программы со своими данными. Физически поток представлен 16 собственными 32-битными регистрами в регистровом файле ядра и номер l (0–31) внутри своей группы. Собственного счётчика команд у потока нет. Поток l использует свой номер, чтобы выбрать свои данные: например, вход x_l.
- **Варп**: группа из 32 потоков с общим счётчиком команд. Все потоки варпа выполняют одну и ту же команду, каждый над своими регистрами. В ядре 2 варпа, всего 64 потока; каждый варп считает один скрытый нейрон.
- **Линия** (lane): физический умножитель-сумматор FP32. Выполняет одну операцию $a \cdot b + c$ за такт для одного потока; сложение и умножение являются частными случаями этой операции. В ядре 4 линии, на кристалле 16.
- **Порция**: 4 потока, которые линии обрабатывают за один такт. Команда варпа из 32 потоков проходит через 4 линии за 8 порций, то есть за 8 тактов.
- **Маска**: 32 бита в команде; бит l равен 1, если поток l выполняет команду. Поток с битом 0 команду пропускает и свои регистры не меняет. Порция, в которой все 4 потока выключены маской, не выдаётся и такт не занимает.

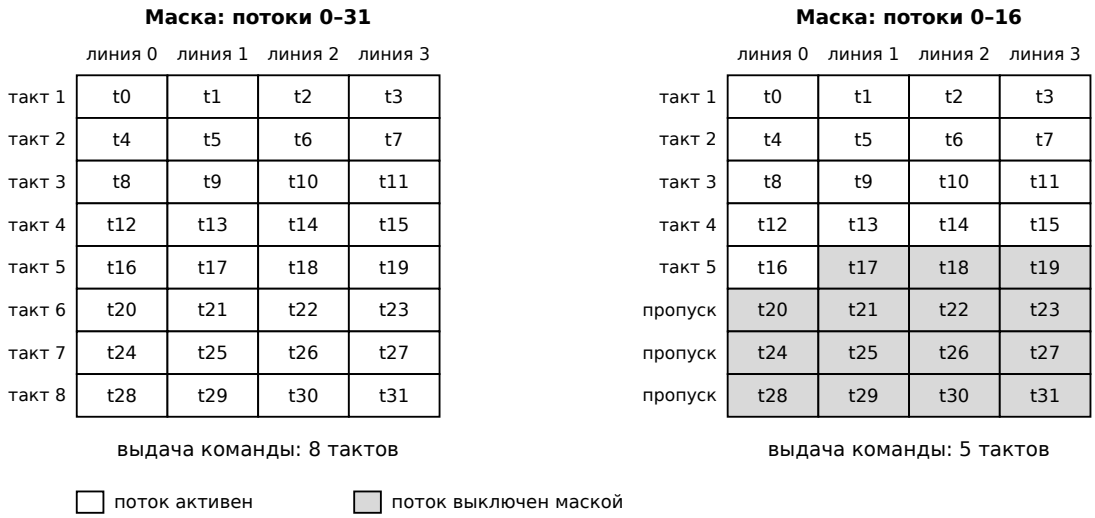


Рисунок 4. Выполнение команды варпа на четырёх линиях

Как читать рисунок 4. Каждая клетка соответствует потоку t0–t31, столбец показывает линию, на которой поток выполняется, строка показывает порцию. Белая клетка: поток включён маской, серая: выключен. Слева от строки указан такт выдачи или «пропуск». Левая часть: команда для всех 32 потоков занимает 8 тактов. Правая часть: маска 0–16 (второй проход по входам нейрона, раздел 6) включает 17 потоков; порции с t20 по t31 пропускаются, команда занимает 5 тактов.

3.5. Организация памяти

Кэшей, внешней оперативной памяти и накопителя у процессора нет. Файл с весами сети хранится на хосте; при конфигурировании веса записываются в локальную память ядер и остаются там до следующей загрузки. Вся память кристалла адресуется явно: программа указывает, из какой памяти и по какому адресу читать. Кэш не нужен, потому что все данные помещаются в локальную память, а порядок обращений известен заранее; локальная память отвечает за 2 такта на каждое обращение.

Таблица 3. Память кристалла

Память	Где	Объём	Что хранит	Кто пишет	Кто читает
Регистровый файл	в каждом PE	4 КиБ	регистры 64 потоков: загруженные веса и входы, частичные суммы	исполнительные блоки PE	исполнительные блоки PE
Local SRAM	в каждом PE	2 КиБ	веса своих двух нейронов, копия входа, вклады варпа 1	IO controller, LSU своего PE	LSU своего PE
Память команд	в каждом PE	2 КиБ	программы двух варпов	IO controller	планировщик PE
ROM сигмоиды	в каждом PE	1028 Б	257 значений σ	записывается при изготовлении	блок SIG
Shared SRAM	одна на кристалл	4 КиБ	два буфера входа, MBOX	IO controller, ядра	IO controller, PE0
Регистры IO controller	в IO controller	32 Б	y_0–y_2, класс, START, DONE	PE0, хост	хост

Таблица 4. Карта локальной памяти ядра PEp

Адрес	Размер	Содержимое
0x000–0x061	98 Б	строка W^1 нейрона 2p: 49 весов BF16
0x080–0x0E1	98 Б	строка W^1 нейрона 2p + 1: 49 весов BF16
0x100–0x10B	12 Б	веса W^2 от нейронов 2p и 2p + 1 к трём выходам: элемент W^2[o][2p + w] по адресу 0x100 + 2 · (2o + w), BF16
0x120–0x181	98 Б	копия входа x: 49 значений BF16
0x1A0–0x1AB	12 Б	вклады варпа 1 в три выхода, FP32
0x1C0–0x7FF	1600 Б	свободно

Таблица 5. Карта общей памяти

Адрес	Размер	Содержимое
0x000–0x061	98 Б	буфер входа A: 49 значений BF16
0x080–0x0E1	98 Б	буфер входа B (пока ядра считают изображение из A, хост заполняет B)
0x100–0x12F	48 Б	MBOX: 4 ядра × 3 частичные суммы FP32; элемент 3p + o хранит сумму ядра p для выхода o
0x200–0xFFF	3584 Б	свободно; используется для обмена между скрытыми слоями в сетях с несколькими скрытыми слоями (раздел 5.4)

Контроллер общей памяти считает записи в область MBOX. Этот счётчик читает команда WAIT ядра PE0, чтобы дождаться частичных сумм всех четырёх ядер.

3.6. Модель вычислений, синхронизация и программирование

Модель вычислений процессора описывается на трёх уровнях: как работает одно ядро, как работают ядра вместе и как процессор программируется.

Таблица 6. Модель вычислений

Уровень	Модель	Что это значит для сети 49-8-3
Ядро	SIMT: одна команда выполняется группой из 32 потоков (варпом); в ядре 2 варпа со своими счётчиками команд	варп считает один нейрон, поток берёт один или два входа нейрона
Кристалл	4 независимых ядра, каждое со своей программой и своей локальной памятью; обмен только через общую память (в классификации Флинна: MIMD из 4 ядер, каждое из которых SIMD-подобное)	ядра считают разные нейроны и обмениваются 12 частичными суммами
Доступ к памяти	загрузка-сохранение: арифметика работает только с регистрами, память читают и пишут отдельные команды LD и ST	каждый вес проходит путь local SRAM, LSU, регистр, линия FMA
Размещение данных	статическое: адреса всех весов, входа и промежуточных значений выбирает хост при конфигурировании; кэшей нет	время расчёта не зависит от данных и одинаково для любого изображения
Управление	команды варпа выполняются по порядку; переходы только LOOP с одинаковым условием для всех потоков варпа; выбор потоков задаёт маска в команде	ветвлений, зависящих от данных, нет; выбор класса делает команда CLS

Синхронизация выполняется тремя механизмами. Сигнал START от IO controller запускает все варпы всех ядер одновременно. Команда BAR синхронизирует два варпа одного ядра: варп, дошедший до BAR, не выдаёт команд, пока второй варп не дойдёт до своей команды BAR. Команда WAIT n синхронизирует ядра между собой через общую память: при каждой записи в MBOX счётчик записей shared SRAM увеличивается на 1, и варп, выполняющий WAIT n, стоит, пока счётчик меньше n. Счётчик обнуляется сигналом START.

Программная модель: одна программа на варп (single program, multiple data). Программа пишется один раз, а различие между потоками задаётся только номером потока l. Номер l участвует в двух местах: в адресе загрузки (адрес = база + (смещение + шаг · l) · размер элемента) и в командах SHFL, которые передают регистры между потоками. Поэтому одна команда LD для 32 потоков загружает 32 разных пикселя, а одна команда FMA выполняет 32 разных умножения.

4. Микроархитектура функциональных узлов (п. 4.3)

4.1. Вычислительное ядро PE

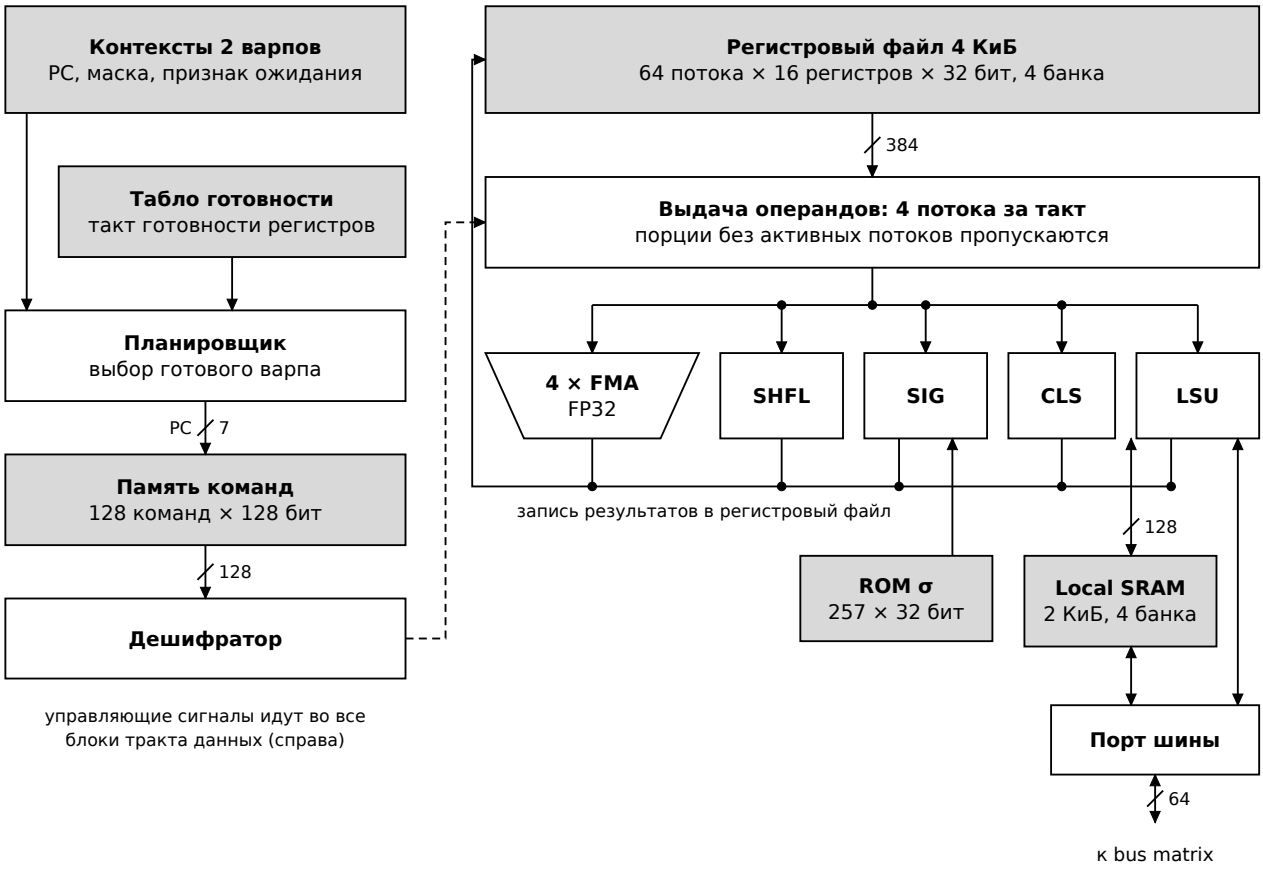


Рисунок 5. Структурная схема вычислительного ядра PE

Как читать рисунок 5. Левый столбец показывает управляющую часть ядра, правая часть показывает тракт данных. Штриховая линия от дешифратора показывает управляющие сигналы; они идут во все блоки тракта данных, на схеме изображены одной линией. Линия «запись результатов» собирает результаты исполнительных блоков и возвращает их в регистровый файл; точки на ней отмечают места подключения блоков. Пересечения линий без точки соединений не означают. Серым залиты блоки, которые хранят данные: контексты варпов, табло готовности, память команд, регистровый файл, постоянная память значений сигмоиды ROM σ и локальная память Local SRAM. Белые прямоугольники обозначают логику: она ничего не хранит между командами, а только вычисляет или передаёт. Трапеция «4 × FMA» обозначает арифметический узел, как принято в схемах Харриса: операнды входят с широкой стороны, результат выходит с узкой. Сплошные стрелки показывают направление движения данных, число у косой черты на линии показывает её ширину в битах: 7 бит адреса команды PC, 128 бит команды, 384 бита операндов (3 операнда × 4 потока × 32 бита), 128 бит между LSU и локальной памятью (4 потока × 32 бита), 64 бита шины. Двусторонняя стрелка означает, что по линии данные идут в обе стороны (чтение и запись).

Таблица 7. Узлы вычислительного ядра

Блок	**Функция**
Контексты 2 варпов	для каждого варпа: счётчик команд PC (7 бит), признак ожидания барьера или WAIT, признак завершения
Табло готовности	для каждого варпа и регистра номер такта, с которого результат можно читать
Планировщик	каждый такт, если тракт выдачи свободен, выбирает варп, у которого все операнды следующей команды готовы; при двух готовых варпах выбирает их по очереди
Память команд	128 команд по 128 бит; программы обоих варпов, точка входа каждого варпа задаётся при конфигурировании
Дешифратор	разбирает поля команды (рисунок 8) в управляющие сигналы
Регистровый файл	64 потока × 16 регистров × 32 бита = 4 КиБ; 4 банка, поток l хранится в банке l mod 4; у банка 3 порта чтения и 1 порт записи, поэтому за такт читаются три операнда для каждого из 4 потоков порции
Выдача операндов	перебирает порции команды, пропуская порции без активных потоков, и подаёт операнды 4 потоков в исполнительный блок: 3 операнда × 4 потока × 32 бита = 384 бита за такт
4 × FMA	4 линии FP32: d = a · b + c с одним округлением; MUL выполняется как a · b + 0, ADD как a · 1 + c; результат через 3 такта после выдачи порции
SHFL	коммутатор: поток l получает регистр потока l + s (сдвиг s) или потока с заданным номером того же варпа
SIG	сигмоида по таблице с линейной интерполяцией (раздел 4.2); один поток за такт; результат через 10 тактов
CLS	выбор класса: максимум среди активных потоков, сравнение с порогом, номер класса в поток 0
LSU	блок загрузки и сохранения: адреса 4 потоков за такт, расширение BF16 до FP32 при загрузке, обращения к local SRAM напрямую, к shared SRAM и регистрам IO controller через порт шины
Local SRAM	2 КиБ, 4 банка по 512 Б со словом 32 бита; за такт 4 элемента FP32 или 4 элемента BF16
ROM σ	257 значений сигмоиды FP32
Порт шины	подключение ядра к bus matrix: запросы LSU и запись в local SRAM со стороны IO controller

Правило выдачи команд. Команда занимает тракт выдачи столько тактов, сколько в ней порций хотя бы с одним активным потоком. Команда одного варпа не может начаться, пока не готовы все её операнды; в это время планировщик выдаёт команду другого варпа. Внутри варпа команды выдаются строго по порядку программы.

4.2. Блок сигмоиды SIG

Экспоненту блок не вычисляет: значение σ берётся из таблицы и уточняется линейной интерполяцией между соседними узлами. Таблица L содержит 257 узлов σ(−8 + k/16), k = 0…256, с шагом 1/16 на отрезке [−8; 8].

$$u = 16\,(z+8), \quad k = \lfloor u \rfloor, \quad a = u - k, \qquad \tilde{\sigma}(z) = L_k + a\,(L_{k+1} - L_k)$$

Обозначения: z: аргумент; u: положение z в единицах шага таблицы; k: номер левого узла; a: доля пути от узла k до узла k + 1 (от 0 до 1); L_k: значение таблицы в узле k; σ̃(z): результат блока. При z ≤ −8 результат равен 0, при z ≥ 8 результат равен 1.

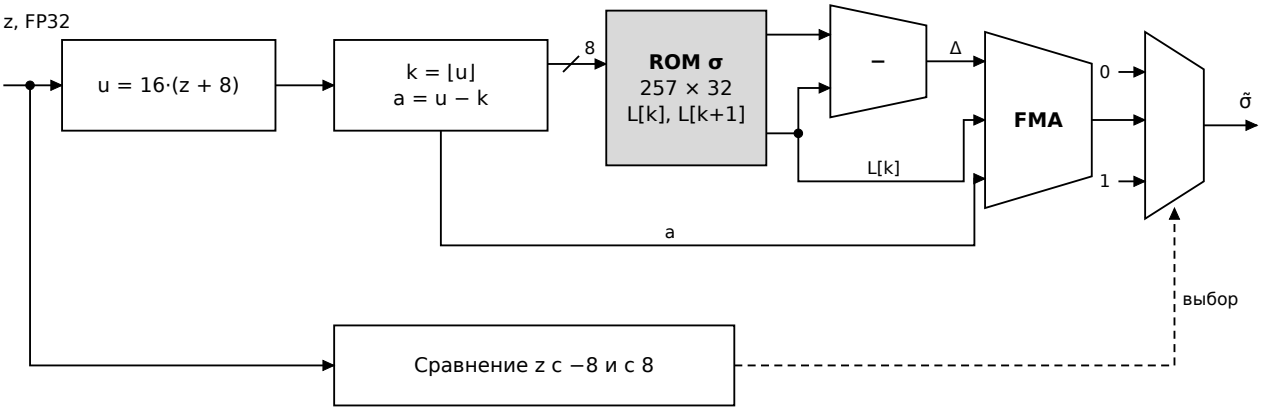


Рисунок 6. Структурная схема блока сигмоиды SIG

Как читать рисунок 6. Данные идут слева направо. Первый блок переводит z в координату таблицы u, второй делит u на целую часть k (8 бит, номер узла) и дробную часть a. ROM выдаёт значения двух соседних узлов. Трапеция «−» вычисляет разность $\Delta = L_{k+1} - L_k$, трапеция FMA вычисляет $a \cdot \Delta + L_k$. Мультиплексор справа выбирает результат интерполяции, константу 0 или константу 1; сигнал выбора (штриховая линия) формирует блок сравнения z с −8 и 8.

Погрешность. Для линейной интерполяции ошибка не превышает $h^2 \cdot \max|\sigma''| / 8$, где h = 1/16 это шаг таблицы, $\max|\sigma''| = \sqrt{3}/18$ это наибольшее значение второй производной сигмоиды:

$$|\tilde{\sigma}(z) - \sigma(z)| \le \frac{h^2 \max|\sigma''|}{8} = \frac{(1/16)^2 \cdot \sqrt{3}/18}{8} \approx 4.70 \cdot 10^{-5}$$

Проверка программой на сетке из 65 535 точек внутри (−8; 8) дала наибольшую ошибку 4.70e-05. За пределами отрезка ошибка насыщения не больше σ(−8) = 3.35e-04. Решение «шум или класс» принимается по порогу 0.5, поэтому такая точность достаточна.

4.3. Bus matrix и контроллер ввода-вывода

Bus matrix имеет 5 ведущих портов (IO controller и 4 ядра) и 6 ведомых: shared SRAM, порты 4 ядер (через них IO controller пишет в локальную память и память команд ядра) и регистры IO controller. Ширина данных 64 бита. Передача занимает 2 такта на арбитраж и адрес и по 1 такту на каждое 64-битное слово. Передачи к разным ведомым идут одновременно; к одному ведомому идут по очереди, порядок назначает арбитр по кругу. Запись входного изображения выполняется широковещательно: одна передача записывает одно и то же изображение во все 4 локальные памяти.

IO controller содержит: приёмник и передатчик последовательного канала; разборщик пакетов; блок прямого доступа к памяти (DMA), который переносит данные пакетов в память кристалла; регистры управления (RESET, START), состояния (DONE) и результата (y_0–y_2, класс). Формат пакетов описан в разделе 5.3.

5. Форматы данных и программирование (п. 4.4)

5.1. Форматы чисел

Таблица 8. Форматы чисел

Данные	Формат в памяти	Формат при вычислении
Вход x (0 или 1)	BF16	FP32
Веса W^1, W^2	BF16	FP32
Выход скрытого нейрона h	округляется до BF16	FP32
Частичные суммы, выходы y	FP32	FP32
Адреса, счётчики, номер класса	целые 32 бита	целые 32 бита

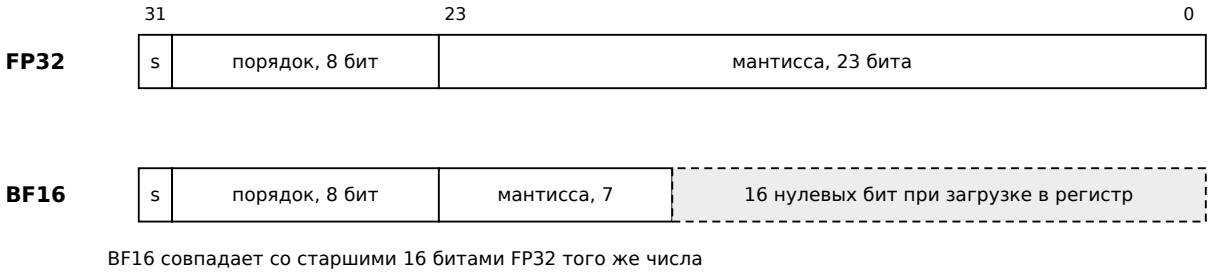


Рисунок 7. Форматы FP32 и BF16

Как читать рисунок 7. Прямоугольники обозначают поля числа, числа над ними указывают номера битов. BF16 имеет тот же знак и тот же 8-битный порядок, что FP32, но только 7 бит мантиссы. Поэтому число BF16 совпадает со старшими 16 битами числа FP32, и при загрузке в регистр LSU просто дописывает 16 нулевых бит (штриховое поле).

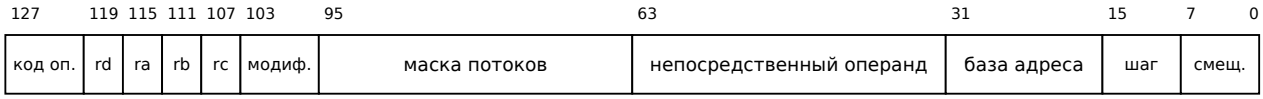
Почему так. BF16 вдвое уменьшает объём весов: 416 весов занимают 832 Б вместо 1664 Б. Относительная ошибка округления веса в BF16 не больше $2^{-8} \approx 0.4$ %. Произведение двух чисел BF16 (по 8 значащих бит) имеет не больше 16 значащих бит и точно помещается в FP32 (24 значащих бита), поэтому умножение ошибки не добавляет. Суммы копятся в FP32: при накоплении 49 слагаемых в BF16 младшие разряды терялись бы на каждом сложении.

Таблица 9. Сравнение форматов хранения весов

Формат	Объём 416 весов	Значащих битов	Диапазон	Решение
FP32	1664 Б	24	как у FP32	не выбран для хранения: вдвое больше памяти и передач, точность избыточна для весов, обученных в double и округляемых всё равно
FP16	832 Б	11	до 65504, порядок 5 бит	не выбран: перевод в FP32 требует отдельного блока перестройки порядка, малые веса теряют точность
BF16	**832 Б**	**8**	**как у FP32**	**выбран: перевод в FP32 дописыванием 16 нулей, переполнение невозможно, ошибка не больше 0,4 %, произведение двух BF16 точно помещается в FP32**
INT8	416 Б	8 (целое)	задаётся масштабом	не выбран: нужны масштабные множители на слой, калибровка, отдельный целочисленный тракт и обратный перевод перед сигмоидой

Итоговое правило: в памяти (local SRAM, shared SRAM, пакеты хоста) веса, вход и выходы скрытого слоя хранятся в BF16; в регистрах и на линиях FMA все вычисления идут в FP32; частичные суммы q, которые ещё будут складываться, хранятся в FP32, чтобы не округлять их дважды.

5.2. Формат команды и система команд



Команда 128 бит; номера битов указаны над полями

Рисунок 8. Формат команды

Как читать рисунок 8. Команда занимает 128 бит; прямоугольники обозначают поля, число над левым краем поля указывает номер старшего бита поля. Поля rd, ra, rb, rc содержат номера регистров результата и операндов (16 регистров, по 4 бита). Поле «модиф.» задаёт пространство памяти (local, shared, IO), формат (BF16 или FP32) и режим SHFL. Маска потоков занимает 32 бита, по биту на поток. Непосредственный операнд содержит константу FP32, сдвиг SHFL, порог CLS или адрес перехода. Адрес элемента для потока l вычисляется как база + (смещение + шаг · l) · размер элемента, где размер равен 2 для BF16 и 4 для FP32.

Таблица 10. Система команд

Команда	Действие для каждого активного потока l
LD.BF16 / LD.F32 rd, адрес	загрузить элемент из local SRAM в rd (BF16 расширяется до FP32)
LD.S rd, адрес	загрузить элемент из shared SRAM через шину
ST.L / ST.S / ST.IO адрес, ra	сохранить ra в local SRAM, shared SRAM или регистр IO controller
MUL rd, ra, rb	rd = ra · rb
FMA rd, ra, rb, rc	rd = ra · rb + rc с одним округлением
ADD rd, ra, rb	rd = ra + rb
SHFL.DOWN rd, ra, s	rd = ra потока l + s
SHFL.IDX rd, ra, k	rd = ra потока k
SIG rd, ra	rd = σ̃(ra)
CVT.BF16 rd, ra	округлить ra до BF16 (к ближайшему, при равенстве к чётному)
CLS rd, ra, порог	поток 0: номер потока с наибольшим ra, или 3, если наибольшее ra меньше порога
IADD rd, ra, imm	целое сложение (адреса и счётчики)
LOOP rc, метка	уменьшить счётчик rc и перейти к метке, если он не ноль; одинаково для всех потоков варпа
BAR	ждать, пока второй варп этого ядра дойдёт до своей команды BAR
WAIT n	ждать, пока в MBOX будет записано n частичных сумм
END	завершить варп

5.3. Описание сети и процесс конфигурирования

Описание сети хранится в файле JSON, который программа лабораторной работы 1 сохраняет после обучения. Веса записываются в исходном формате double; округление до BF16 выполняет хост.

```
{
  "shape": [49, 8, 3],                 // входов, нейронов скрытого слоя, выходов
  "activation": "sigmoid",
  "bias": false,
  "classes": ["circle", "square", "triangle"],
  "noise_threshold": 0.5,
  "W1": [[w00, w01, ..., w0_48], ..., [w70, ..., w7_48]],   // 8 строк по 49 чисел
  "W2": [[...8 чисел...], [...], [...]]                      // 3 строки по 8 чисел
}
```

Таблица 11. Формат пакета между хостом и IO controller

Поле пакета	Размер	Значение
Тип	1 Б	RESET, LOAD (запись в память), IMAGE, RESULT
Получатель	1 Б	память команд PEp, local SRAM PEp, shared SRAM
Адрес	2 Б	адрес начала записи в памяти получателя
Длина	2 Б	число байт данных
Данные	до 2048 Б	программа, веса или изображение
Контрольная сумма	1 Б	сумма байт пакета по модулю 256

Процесс конфигурирования и запуска показан на рисунке 9. Хост делает всю подготовку: проверяет размеры слоёв, округляет веса до BF16, раскладывает строки W^1 и столбцы W^2 по ядрам по правилу раздела 6.1 и формирует программы варпов с конкретными адресами. Процессор получает уже готовые программы и данные.

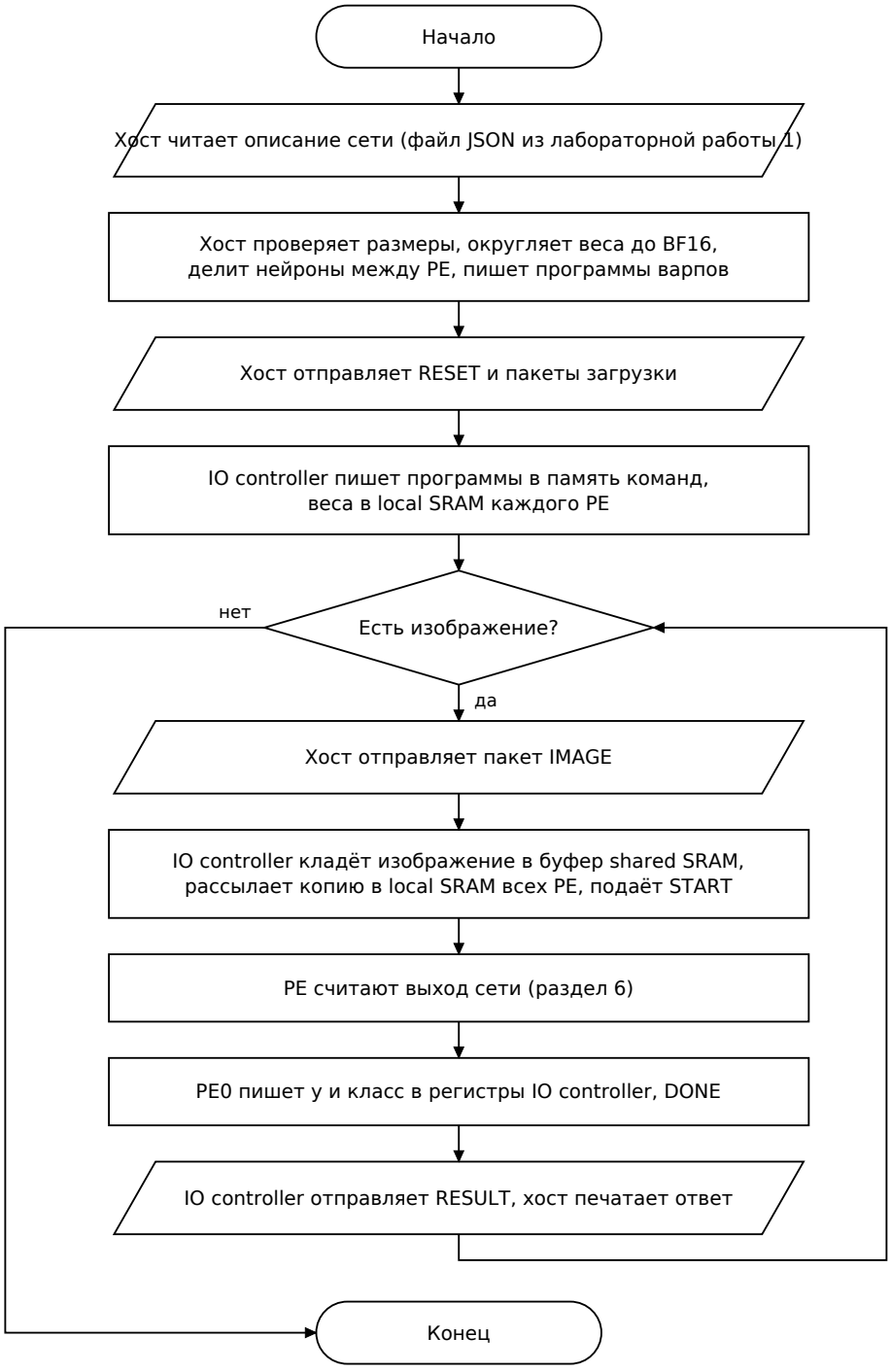


Рисунок 9. Алгоритм конфигурирования процессора и обработки изображений

5.4. Расчёт сетей другой структуры

Процессор не привязан к сети 49-8-3: структура задаётся полем shape описания, а программы варпов строит хост. Правила распределения для сети с N входами и несколькими слоями:

- нейроны каждого слоя делятся между 4 ядрами поровну; в ядре варп считает один нейрон, а закончив его, переходит к следующему нейрону своего ядра (команда LOOP);
- взвешенная сумма нейрона с N входами считается за N/32 проходов, округлённое вверх: в проходе r поток l берёт вход с номером l + 32r; затем 5 шагов свёртки;
- между двумя слоями каждому ядру нужен весь выходной вектор предыдущего слоя, а посчитало ядро только свою четверть. Поэтому каждый варп записывает выход своего нейрона в общую память, ядра ждут, пока будут записаны все выходы слоя, и каждое ядро загружает весь вектор в свою локальную память;
- последний слой считается так же, как выходной слой сети 49-8-3: каждое ядро считает вклад своих нейронов в каждый выход, PE0 складывает вклады.

Таблица 12. Ограничения на структуру сети

Ресурс	Ограничение	Пример
Local SRAM 2 КиБ	веса одного ядра и копия входа занимают до 1900 Б, около 950 весов BF16	49-32-3: 8 нейронов на ядро, 416 весов, 832 Б, помещается
Память команд 128 команд	без LOOP программа растёт с числом нейронов на ядро	49-8-3: 60 команд в PE0; 49-32-3 требует LOOP
Регистровый файл	16 регистров на поток	достаточно для любой ширины слоя: веса загружаются по мере счёта
Shared SRAM 4 КиБ	выход одного промежуточного слоя занимает до 3584 Б	до 896 нейронов в слое при хранении FP32

Число проходов по входам нейрона с N входами и число нейронов на варп для слоя из K нейронов:

$$R = \left\lceil \frac{N}{32} \right\rceil, \qquad m = \left\lceil \frac{K}{8} \right\rceil$$

Здесь N: число входов нейрона (размер предыдущего слоя), 32: число потоков варпа, R: число проходов, K: число нейронов слоя, 8: число варпов на кристалле (4 ядра по 2 варпа), m: число нейронов, которые считает один варп по очереди (команда LOOP повторяет тело программы m раз). Угловые скобки в формуле означают округление вверх до целого.

Пример: сеть 49-16-8-3 (два скрытых слоя).

Таблица 13. Распределение сети 49-16-8-3; N, K, R, m определены выше

Слой	N	K	R	m	Весов на ядро	Обмен после слоя
Скрытый 1	49	16	2	2	$4 \cdot 49 = 196$	16 выходов FP32 в shared SRAM, WAIT 16, каждое ядро загружает все 16
Скрытый 2	16	8	1 (потоки 0–15)	1	$2 \cdot 16 = 32$	вклады в 3 выхода, как в сети 49-8-3
Выходной	8	3	свёртка вкладов в PE0	нет	$2 \cdot 3 = 6$	y и класс в IO controller

Всего на ядро 196 + 32 + 6 = 234 веса BF16, то есть 468 Б из 2048 Б local SRAM, сеть помещается. Общий алгоритм для любой сети: хост проверяет ограничения таблицы выше, для каждого слоя вычисляет R и m, раскладывает строки матрицы весов по варпам по кругу (нейрон k достаётся варпу k mod 8), генерирует для каждого слоя тело программы из раздела 6.2 с R проходами и LOOP на m нейронов, после каждого слоя, кроме последнего, вставляет запись выходов в shared SRAM, WAIT и загрузку вектора, после последнего слоя вставляет свёртку вкладов в PE0, SIG и CLS.

5.5. Что программируется и как

Таблица 14. Программируемые и фиксированные параметры процессора

Что	Кто задаёт	Как
Структура сети (число слоёв и нейронов)	оператор	поле shape описания JSON
Веса	оператор (результат обучения)	поля W1, W2, ... описания; хост округляет до BF16
Классы и порог шума	оператор	поля classes и noise_threshold; порог попадает в поле непосредственного операнда команды CLS
Распределение нейронов по ядрам и варпам	генератор программ на хосте	правила раздела 5.4
Программы варпов	генератор программ на хосте или вручную на ассемблере	система команд таблицы раздела 5.2; программа загружается пакетом LOAD в память команд ядра
Адреса данных в памяти	генератор программ	поля база, шаг, смещение в каждой команде LD и ST
Функция активации	аппаратура	таблица σ в постоянной памяти ROM σ; поддерживается только сигмоида. Если заменить ROM на записываемую память того же размера и загружать её пакетом LOAD, процессор сможет считать любую гладкую функцию активации (например, гиперболический тангенс) без изменения остальных блоков
Число ядер, варпов, потоков, регистров, объёмы памяти	аппаратура	не меняются

6. Алгоритм расчёта выхода сети (п. 4.5)

6.1. Распределение работы

Ядро PEp (p = 0…3) отвечает за скрытые нейроны 2p и 2p + 1. Варп w (0 или 1) ядра PEp считает нейрон n = 2p + w. Внутри варпа поток l в первом проходе умножает вход x_l, во втором проходе вход x_{l+32} (только потоки 0–16, так как вход 32 + 16 = 48 последний).

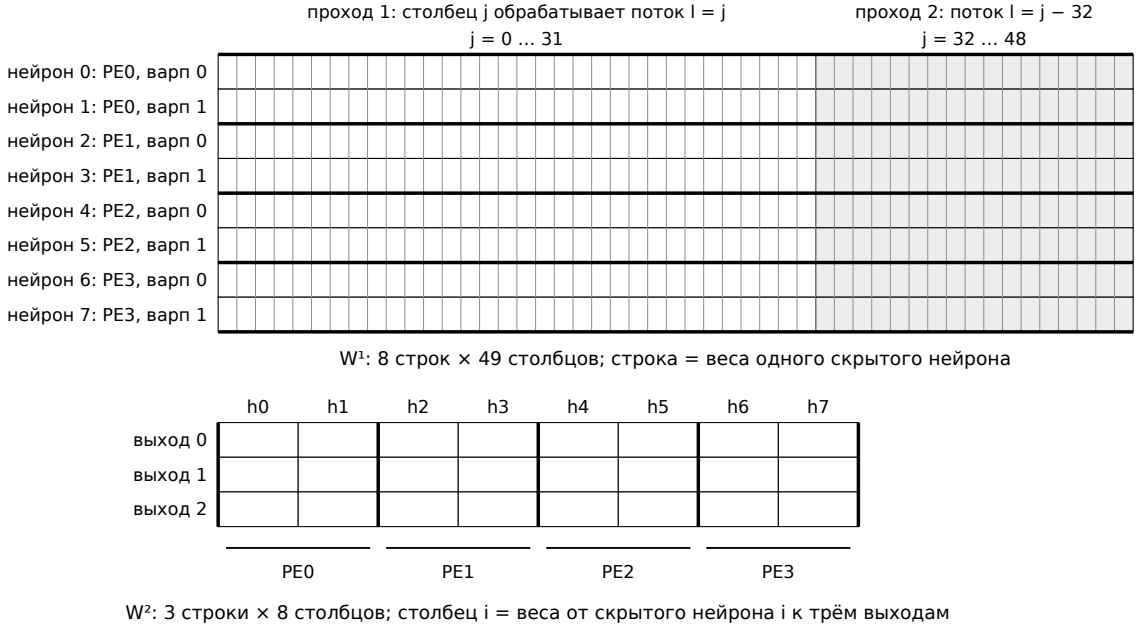


Рисунок 10. Распределение весов между ядрами, варпами и потоками

Как читать рисунок 10. Верхняя сетка изображает матрицу W^1: строка содержит веса одного скрытого нейрона, столбец содержит веса от одного пикселя. Слева у каждой строки указано, какое ядро и какой варп её считают; толстые линии отделяют строки разных ядер. Белая часть строки обрабатывается в первом проходе, серая во втором. Нижняя сетка изображает матрицу W^2: строка соответствует выходу, столбец скрытому нейрону; ядро PEp хранит столбцы 2p и 2p + 1 (подписи под сеткой).

Выходной слой считается по скрытым нейронам: варп, посчитавший h_n, сразу умножает его на три веса своего столбца W^2. Получаются вклады нейрона n в три выхода. Ядро складывает вклады двух своих варпов, PE0 складывает суммы четырёх ядер:

$$c_{o,n} = w^{(2)}_{on}\, h_n, \qquad q_{p,o} = c_{o,2p} + c_{o,2p+1}, \qquad z_o = (q_{0,o} + q_{2,o}) + (q_{1,o} + q_{3,o}), \qquad y_o = \tilde{\sigma}(z_o)$$

Обозначения: $c_{o,n}$: вклад скрытого нейрона n в выход o; $q_{p,o}$: частичная сумма ядра p для выхода o; z_o: полная взвешенная сумма выхода o. Скрытые выходы h между ядрами не пересылаются: каждое ядро использует только свои.

6.2. Программа варпа

Ниже программа варпа 0 ядра PE0 (нейрон 0). Справа от команды указана маска потоков. Варп 1 выполняет те же команды 1–21 для нейрона 1, затем сохраняет свои вклады в local SRAM (ST.L), проходит барьер BAR и завершается.

```
 #  команда                          маска    что делает
 1  LD.BF16  r1, X[l]                 0-31     вход x[l]
 2  LD.BF16  r2, W1[0][l]             0-31     вес нейрона 0 от пикселя l
 3  MUL      r0, r2, r1               0-31     acc = w * x
 4  LD.BF16  r1, X[l+32]              0-16     второй проход
 5  LD.BF16  r2, W1[0][l+32]          0-16
 6  FMA      r0, r2, r1, r0           0-16     acc = acc + w * x
 7  SHFL.DOWN r3, r0, 16              0-15     свёртка: взять acc потока l+16
 8  ADD      r0, r0, r3               0-15
 9  SHFL.DOWN r3, r0, 8               0-7
10  ADD      r0, r0, r3               0-7
11  SHFL.DOWN r3, r0, 4               0-3
12  ADD      r0, r0, r3               0-3
13  SHFL.DOWN r3, r0, 2               0-1
14  ADD      r0, r0, r3               0-1
15  SHFL.DOWN r3, r0, 1               0
16  ADD      r0, r0, r3               0        у потока 0 полная сумма нейрона
17  SIG      r4, r0                   0        h = sigma(сумма)
18  CVT.BF16 r4, r4                   0        округлить h до BF16
19  SHFL.IDX r4, r4, 0                0-2      раздать h потокам 0-2
20  LD.BF16  r5, W2[l][0]             0-2      вес от нейрона 0 к выходу l
21  MUL      r6, r5, r4               0-2      вклад c в выход l
22  BAR                                       ждать варп 1
23  LD.F32   r7, C1[l]                0-2      вклад варпа 1 (нейрон 1)
24  ADD      r6, r6, r7               0-2      q[0][l]
25  ST.S     MBOX[0+l], r6            0-2      в общую память
26  WAIT     4                                 ждать суммы всех 4 ядер (только PE0)
27  LD.S     r7, MBOX[l]              0-11     поток l = 3p+o получает q[p][o]
28  SHFL.DOWN r8, r7, 6               0-5
29  ADD      r7, r7, r8               0-5      q[p][o] + q[p+2][o]
30  SHFL.DOWN r8, r7, 3               0-2
31  ADD      r7, r7, r8               0-2      z[l] = полная сумма выхода l
32  SIG      r9, r7                   0-2      y[l]
33  CLS      r10, r9, 0.5             0-2      класс в поток 0
34  ST.IO    OUT.Y[l], r9             0-2
35  ST.IO    OUT.CLASS, r10           0
36  END
```

Порядок для ядер PE1–PE3 тот же без команд 26–35: после записи частичной суммы в MBOX варп 0 выполняет END. Объём программ: PE0 60 команд, PE1–PE3 по 50 команд из 128 возможных.

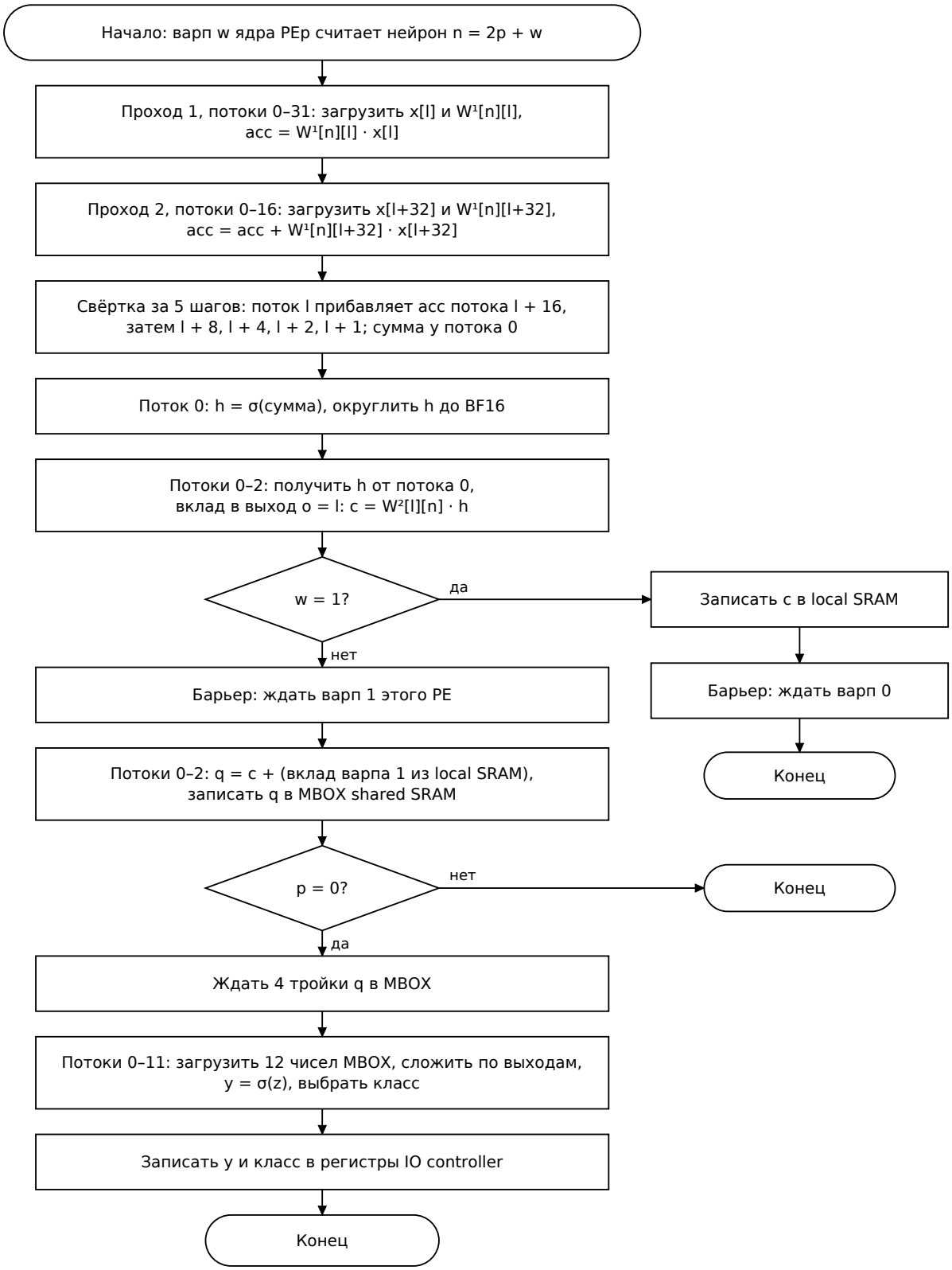


Рисунок 11. Алгоритм варпа

6.3. Пример: изображение круга

Эталонный круг из обучающей выборки лабораторной работы 1: горят пиксели 9, 10, 11, 15, 19, 22, 26, 29, 33, 37, 38, 39 (строки 1–5). Рассмотрим варп 0 ядра PE0, нейрон 0; w_j: вес этого нейрона от пикселя j.

- Поток 9: в первом проходе j = 9, пиксель горит, acc = w_9; во втором проходе j = 41, пиксель не горит, acc не меняется. Итог потока: w_9.
- Поток 5: j = 5 не горит, acc = 0; j = 37 горит, acc = w_{37}.
- Поток 20: j = 20 не горит, acc = 0; во втором проходе поток выключен маской.
- Свёртка, шаг 1: потоки 0–15 прибавляют acc потоков 16–31. Например, поток 10 получает $w_{10} + w_{26}$, поток 3 получает $0 + w_{19}$. Через 5 шагов у потока 0 сумма весов всех 12 горящих пикселей: $z = w_9 + w_{10} + \ldots + w_{39}$.
- Поток 0 вычисляет $h_0 = \sigma(z)$, потоки 0–2 умножают h_0 на $W^2[0][0]$, $W^2[1][0]$, $W^2[2][0]$ и получают вклады нейрона 0 в выходы «круг», «квадрат», «треугольник».
- PE0 складывает вклады всех 8 нейронов в каждый выход, применяет сигмоиду и выбирает класс; для этого изображения сеть из лабораторной работы 1 отвечает «круг».

6.4. Диаграмма последовательности

Рисунок ниже показывает весь путь одного изображения через процессор: от пакета хоста до ответа в консоли. Все узлы структурной схемы задействованы и показаны отдельными столбцами.

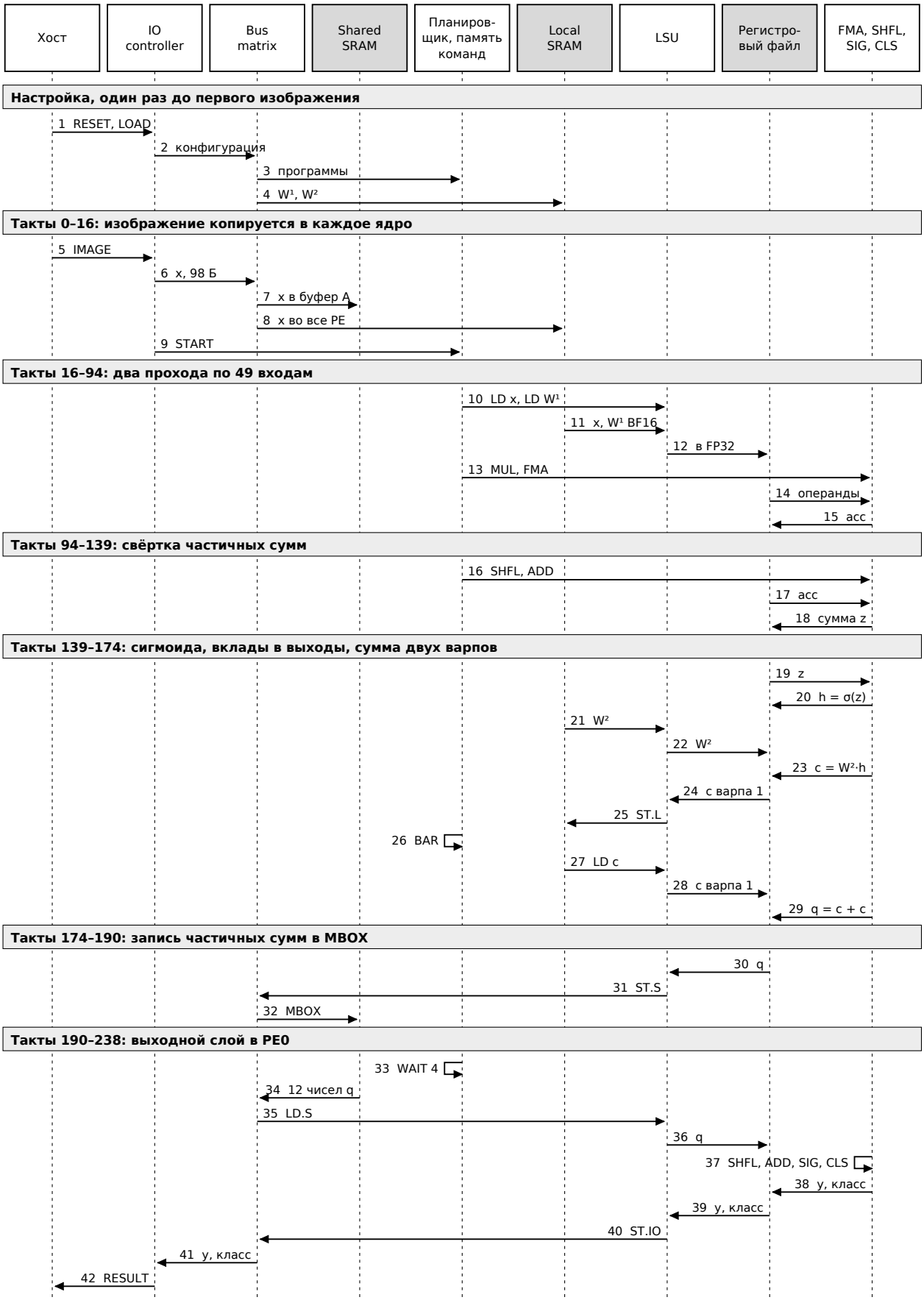


Рисунок 12. Диаграмма последовательности расчёта одного изображения

Как читать рисунок 12. Каждый столбец соответствует узлу процессора, его имя в прямоугольнике сверху; серые прямоугольники обозначают память, белые логику, как на остальных схемах. Время идёт сверху вниз. Штриховая вертикальная линия под узлом показывает, что узел существует всё время расчёта. Горизонтальная стрелка обозначает передачу: данные или команда идут от узла в начале стрелки к узлу, на который указывает остриё. Число у стрелки является номером передачи, текст рядом показывает, что передаётся. Стрелка, которая выходит из узла и в него же возвращается, обозначает действие внутри узла (ожидание или последовательность операций). Серые полосы во всю ширину делят расчёт на этапы и указывают их такты по модели раздела 7. Пересечение стрелки со штриховой линией узла, через который она проходит, не означает передачу через этот узел. Для простоты показано одно ядро: остальные ядра выполняют передачи 10–31 в те же такты.

Таблица 15. Этапы диаграммы последовательности

Передачи	Такты	Что происходит
1–4	до первого и зображения	Хост отправляет RESET и пакеты LOAD. IO controller через bus matrix записывает программы варпов в память команд каждого ядра, а строки W^1 и веса W^2 в local SRAM того ядра, которому они назначены (раздел 3.5).
5–9	0–16	Хост отправляет пакет IMAGE: 49 пикселей по 2 байта BF16. IO controller одной передачей по шине (2 такта на захват и адрес и 13 тактов на 13 слов по 64 бита) записывает изображение в буфер A shared SRAM и одновременно в local SRAM всех четырёх ядер. На такте 16 IO controller подаёт START.
10–15	16–94	Планировщик каждый такт выбирает варп, у которого готовы операнды следующей команды, и читает команду из памяти команд по счётчику PC этого варпа; дешифратор формирует управляющие сигналы. LD: LSU читает x_l и $W^1[n][l]$ из local SRAM, по 4 потока за такт, и пишет их в регистры, расширив BF16 до FP32. MUL и FMA: выдача операндов подаёт по 4 потока за такт на 4 линии FMA, результат acc возвращается в регистр. Первый проход 32 потока, 8 тактов на команду; второй проход 17 потоков, 5 тактов на команду. Варпы 0 и 1 выдают команды по очереди.
16–18	94–139	Свёртка: SHFL передаёт acc потока l + s потоку l, ADD складывает; s = 16, 8, 4, 2, 1. Через 5 шагов поток 0 хранит сумму z_n нейрона.
19–29	139–174	SIG вычисляет $h_n = \sigma(z_n)$ по таблице ROM σ, CVT округляет h до BF16, SHFL раздаёт h потокам 0–2. LSU загружает три веса W^2, линии FMA считают вклады c. Варп 1 сохраняет свои вклады в local SRAM и проходит BAR; варп 0 после BAR загружает их и складывает со своими: q = c(варп 0) + c(варп 1).
30–32	174–190	Варп 0 каждого ядра записывает 3 числа q (FP32) через LSU и шину в MBOX shared SRAM. Шина одна, ядра пишут по очереди, по 4 такта. PE1–PE3 после этого завершаются.
33–42	190–238	Варп 0 ядра PE0 ждёт на WAIT 4, пока счётчик записей MBOX станет равен 4, загружает 12 чисел q через шину (LD.S), складывает их по выходам (SHFL и ADD), вычисляет $y_o = \sigma(z_o)$, выбирает класс (CLS) и записывает y_0, y_1, y_2 и класс в регистры IO controller. IO controller отправляет хосту пакет RESULT, хост выводит ответ в консоль.

7. Оценка времени вычисления выхода сети (п. 4.6)

Время считается от начала рассылки изображения, уже принятого в буфер общей памяти, до записи класса в регистр IO controller. Передача изображения с хоста по последовательному каналу и загрузка конфигурации не входят: они зависят от канала, а не от процессора. Задержки узлов приведены в таблице 16; по ним программа tools/processor_timing_model.py пошагово выдаёт команды всех варпов и передачи по шине.

Таблица 16. Задержки узлов; P: число порций команды, в которых есть активный поток (от 1 до 8)

Операция	**Занимает тракт выдачи**	**Результат готов через**
LD, ST в local SRAM	P тактов	P + 2
MUL, FMA, ADD	P тактов	P + 3
SHFL, CVT	P тактов	P + 1
SIG	по такту на активный поток (n)	n + 9
CLS	1 такт	3
Передача по шине	не занимает	2 + число 64-битных слов
BAR, WAIT, END	1 такт	нет результата

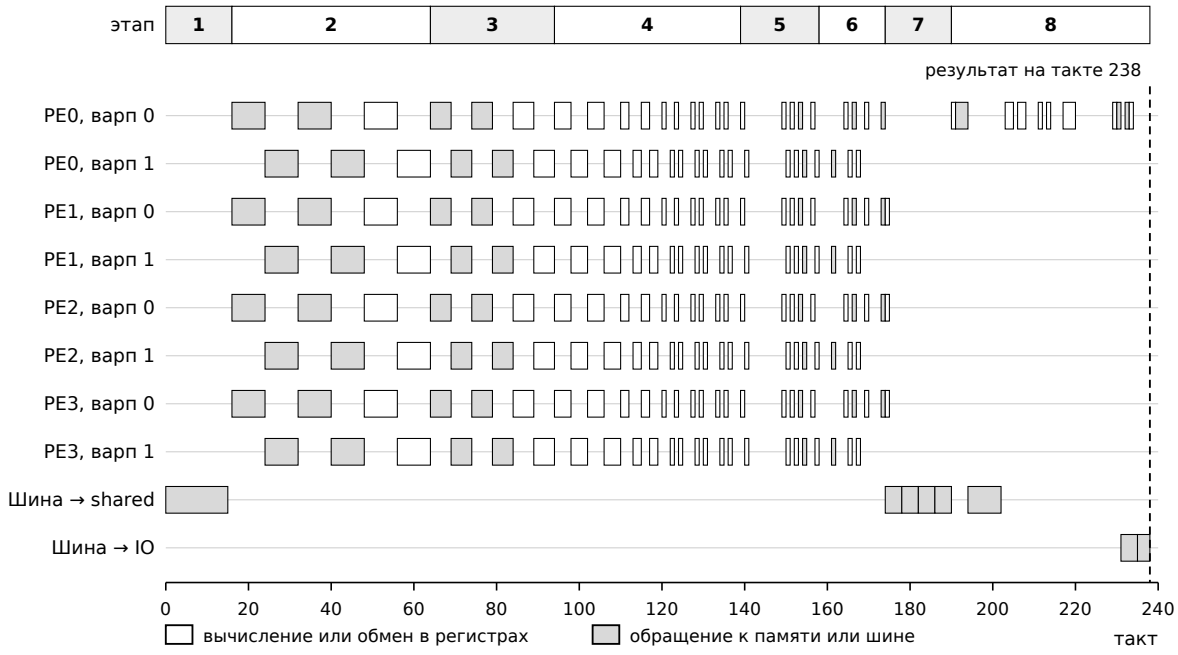


Рисунок 13. Временная диаграмма расчёта одного изображения

Как читать рисунок 13. По горизонтали отложены такты. Строки PE0–PE3 показывают команды каждого варпа каждого ядра: прямоугольник обозначает команду, его ширина равна числу тактов, которые она занимает тракт выдачи; белый прямоугольник обозначает вычисление или обмен между потоками, серый обращение к памяти. Ядра работают одновременно и до такта 174 выполняют одинаковые программы над разными нейронами, поэтому их строки совпадают; после записи частичных сумм PE1–PE3

завершаются, а PE0 досчитывает выходной слой. Два варпа одного ядра никогда не занимают тракт выдачи в один и тот же такт: прямоугольники варпа 0 и варпа 1 чередуются. Две нижние строки показывают передачи по шине к общей памяти и к IO controller. Пустое место в строке означает, что варп ждёт: операнды не готовы или тракт занят другим варпом. Номера вверху соответствуют этапам из таблицы 17.

Таблица 17. Этапы расчёта

Этап	Такты	Что происходит
1	0–16	IO controller рассылает изображение в local SRAM всех ядер (15 тактов) и подаёт START
2	16–64	первый проход: загрузка x и W^1, умножение, 32 потока, 3 команды на варп
3	64–94	второй проход, потоки 0–16: команды по 5 тактов вместо 8
4	94–139	свёртка 32 частичных сумм в одну: 5 пар SHFL + ADD
5	139–158	сигмоида, округление h, вклады в три выхода
6	158–174	сложение вкладов двух варпов внутри ядра, отправка q в MBOX
7	174–190	4 записи в MBOX по шине, по очереди
8	190–238	PE0: загрузка 12 сумм, сложение, сигмоида выходов, выбор класса, запись в IO controller

Таблица 18. Время вычисления выхода сети

Вариант	Тактов	Время при 100 МГц
Без пропуска пустых порций (каждая команда 8 тактов)	515	5.15 мкс
С пропуском, варпы по очереди без чередования	297	2.97 мкс
С пропуском и чередованием варпов (принятый)	**238**	**2.38 мкс**

Пропуск пустых порций сокращает время в 2.2 раза: на этапах 3–6 и 8 большинство команд выполняется для 1–17 потоков. Чередование варпов экономит 59 тактов: пока один варп ждёт результата предыдущей команды (3 такта после MUL, FMA, ADD и 10 тактов после SIG), выдаётся команда другого варпа.

7.1. Способы сокращения задержки

Задержка (время от START до готового класса) сокращена способами из таблицы ниже. В столбце «Эффект» пометка «модель» означает, что значение получено моделью tools/processor_timing_model.py; пометка «оценка» означает расчёт по задержкам таблицы 16 без запуска модели.

Таблица 19. Способы сокращения задержки

Способ	Что устраняет	Эффект
Распределение по нейронам между 4 ядрами и по входам между 32 потоками	последовательный счёт 392 умножений скрытого слоя	каждое ядро выполняет 104 умножения с накоплением, по 4 за такт
Пропуск порций без активных потоков	пустые такты в командах для 1–17 потоков	515 → 238 тактов (модель)
Чередование двух варпов	простой тракта выдачи, пока варп ждёт результат	297 → 238 тактов (модель)
Веса заранее в local SRAM каждого ядра	передачу 832 Б весов по общей шине при каждом изображении	не меньше 104 + 2 · 4 = 112 тактов шины на изображение (оценка)
Широковещательная запись изображения во все ядра	4 отдельные передачи изображения	15 тактов вместо 4 · 15 = 60 (оценка)
Хранение входа в BF16	передачу 196 Б в FP32	15 тактов вместо 2 + 25 = 27 (оценка)
Свёртка деревом через SHFL	31 последовательное сложение, каждое ждёт предыдущее	45 тактов вместо не меньше 31 · 4 = 124 (оценка)
Обмен частичными суммами вместо выходов скрытого слоя	рассылку 8 значений h во все ядра и одну общую синхронизацию	по шине 4 записи по 12 Б; обмен h потребовал бы ещё 4 чтений и ожидания всех ядер
Сигмоида по таблице с интерполяцией	вычисление экспоненты и деления	результат через 10 тактов, 1 такт выдачи на поток
Конвейерные исполнительные блоки	ожидание окончания команды перед выдачей следующей	новая команда может выдаваться каждый такт
Отсутствие кэшей и внешней памяти	промахи и непредсказуемые задержки	время одинаково для любого изображения

8. Загрузка вычислительных ядер (п. 4.7)

Таблица 20. Загрузка ядер при расчёте одного изображения

Показатель	PE0	PE1	PE2	PE3
Скрытые нейроны	0, 1	2, 3	4, 5	6, 7
Умножений с накоплением (MAC)	104	104	104	104
Сложений свёртки и сборки сумм	74	65	65	65
Выполненных команд	58	48	48	48
Такт окончания работы	234	175	175	175
Тактов выдачи команд	143	128	128	128
Занятость тракта выдачи за 222 тактов	64.4 %	57.7 %	57.7 %	57.7 %
Загрузка 4 линий FMA	20.0 %	19.0 %	19.0 %	19.0 %

Определения. Занятость тракта выдачи: доля тактов от START до готового результата, в которые ядро выдаёт команду. Загрузка линий: число операций потоков на линиях FMA (умножений и сложений), делённое на 4 линии и на те же 222 тактов.

Основная работа распределена поровну: каждое ядро выполняет 104 умножения с накоплением (98 в скрытом слое и 6 в выходном) и одинаковую программу до записи

частичных сумм. PE0 дополнительно складывает частичные суммы ядер, вычисляет 3 сигмоиды выходов и выбирает класс: это 9 сложений, около 1 % арифметических операций сети, но 59 тактов времени, в которые PE1–PE3 уже свободны.

Линии FMA загружены примерно на 20 %, потому что большая часть команд приходится на загрузки из памяти, свёртку и сигмоиду, а свёртка и выходной слой используют от 1 до 16 потоков из 32. Это свойство модели SIMT на сети такого размера: у нейрона всего 49 входов, и на каждый поток приходится 1–2 слагаемых, поэтому свёртка 32 частичных сумм занимает больше тактов выдачи, чем сами умножения.

9. Выводы

1. Разработан процессор из 4 SIMT-ядер с общей памятью 4 КиБ, локальной памятью 2 КиБ у каждого ядра, коммутатором шин и контроллером ввода-вывода. Выход сети 49-8-3 вычисляется за 238 тактов (2.38 мкс при частоте 100 МГц).

2. Скрытый слой распределён по нейронам: по 2 нейрона на ядро, по 1 нейрону на варп; 49 входов нейрона распределены между 32 потоками варпа. Выходной слой считается по вкладам скрытых нейронов, поэтому выходы скрытого слоя между ядрами не пересылаются; ядра обмениваются только 12 частичными суммами.

3. Каждое ядро выполняет 104 умножения с накоплением; программы ядер до обмена частичными суммами одинаковы. Завершающие сложения, сигмоиды выходов и выбор класса выполняет PE0.

4. Пропуск порций без активных потоков сокращает время с 515 до 238 тактов; чередование двух варпов дополнительно сокращает время с 297 до 238 тактов.

5. Веса хранятся в BF16 (832 Б на всю сеть), вычисления выполняются в FP32. Сигмоида вычисляется по таблице из 257 значений с линейной интерполяцией, ошибка внутри $[-8; 8]$ не превышает $4.7 \cdot 10^{-5}$.

6. Структура сети задаётся описанием JSON; хост раскладывает веса по ядрам и строит программы варпов. Сети с несколькими скрытыми слоями считаются с обменом выходами слоёв через общую память.

7. Загрузка линий FMA около 20 %: для сети с 49 входами на нейрон варп из 32 потоков тратит больше тактов на свёртку частичных сумм, чем на умножения.